

# E-Commerce Web Automation Framework

## CAPSTONE ASSIGNMENT 02

**Training Program:** WIPRO Python Automation Course

**Application Under Test:** TutorialsNinja Demo:

<https://tutorialsninja.com/demo/>

**Technology:** Python 3.10.0 | Selenium 4.48.0 | Pytest | Page Object Model

**Name:** Tamaghna Dhar

**Enrolment no.:** 12023002001241

**Stream:** CSE

**Campus:** IEM Newtown

## 1. Objective

---

The objective of this project is to design and implement a scalable, maintainable, and reusable web test automation framework for an E-Commerce web application. The framework automates the complete end-to-end user journey, from login to product search, cart management, and logout using industry-standard tools and best practices.

The project demonstrates the following key concepts:

- **Page Object Model (POM) design pattern**
- **Data-driven testing using CSV files**
- **Positive and negative test case design**
- **Automated screenshot capture and HTML reporting**
- **Cross-browser test execution (Chrome & Firefox)**

## 2. Tools & Technologies

---

| Tool / Technology       | Version  | Purpose                                    |
|-------------------------|----------|--------------------------------------------|
| Python                  | 3.10.0   | Core programming language                  |
| Selenium WebDriver      | 4.48.0   | Browser automation library                 |
| Pytest                  | Latest   | Test runner, fixtures, assertions          |
| pytest-html             | Latest   | Generates self-contained HTML test reports |
| WebDriver Manager       | Latest   | Auto-downloads browser drivers             |
| Google Chrome / Firefox | Latest   | Target browsers for test execution         |
| CSV (Python built-in)   | Built-in | Stores and reads data-driven test inputs   |
| Python logging          | Built-in | Generates timestamped runtime log files    |
| Git                     | Latest   | Version control                            |

## 3. Application Under Test

---

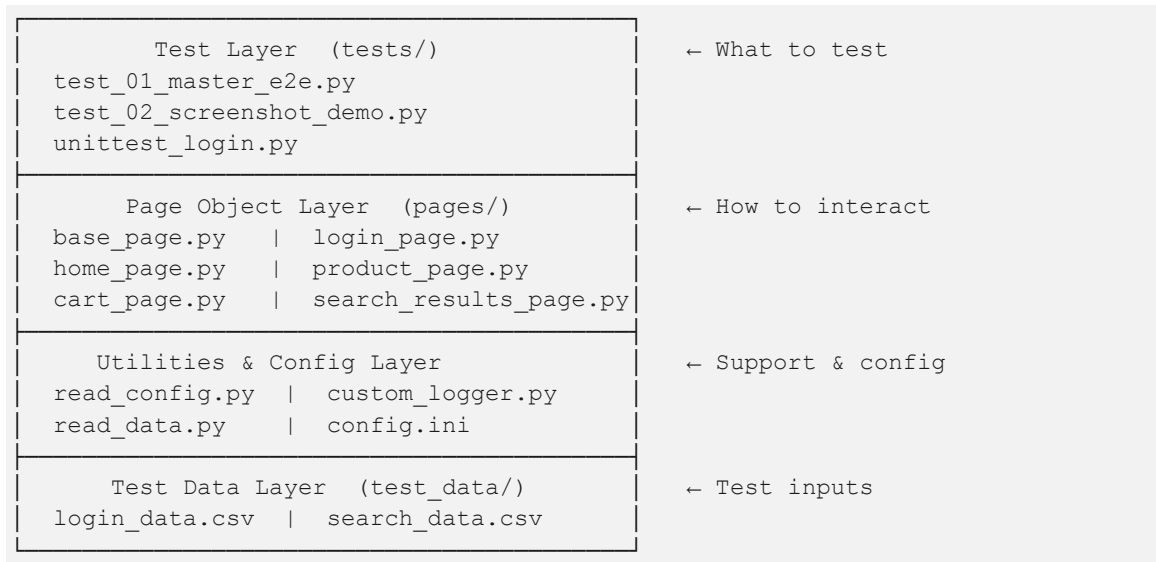
TutorialsNinja is a publicly available e-commerce demo application used for practice and training purposes.

- Base URL: <https://tutorialsninja.com/demo/>
- Features tested: User authentication, product search, product detail page, shopping cart, checkout navigation, logout

## 4. Project Architecture

---

The project follows a layered architecture based on the Page Object Model (POM) design pattern:



### 4.1 Folder Structure

|                               |                                                |
|-------------------------------|------------------------------------------------|
| ecommerce-selenium-framework/ |                                                |
| ├─ config/config.ini          | → Base URL, browser, credentials, timeouts     |
| ├─ pages/                     | → Page Object classes                          |
| ├─ tests/                     | → Test scripts                                 |
| ├─ utilities/                 | → Logger, config reader, CSV reader            |
| ├─ test_data/                 | → CSV files for data-driven testing            |
| ├─ screenshots/               | → Auto-captured screenshots                    |
| ├─ reports/                   | → HTML test report output                      |
| ├─ logs/                      | → Timestamped log files                        |
| ├─ conftest.py                | → Pytest fixtures (driver setup, failure hook) |
| ├─ pytest.ini                 | → Pytest configuration                         |
| ├─ requirements.txt           | → Python package dependencies                  |
| └─ run_demo.bat               | → One-click Windows demo runner                |

## 5. Design Pattern — Page Object Model (POM)

---

Page Object Model (POM) is a design pattern in test automation where each web page is represented as a separate Python class. The class contains:

- Locators: CSS selectors, XPaths, and IDs for UI elements on that page
- Methods: Actions that can be performed on the page (login, search, add to cart, etc.)

## 5.1 Benefits of POM

| Benefit                | Description                                                    |
|------------------------|----------------------------------------------------------------|
| Reusability            | Page methods can be reused across multiple test files          |
| Maintainability        | If a locator changes, it only needs to be updated in one place |
| Readability            | Test scripts read like plain English actions                   |
| Separation of Concerns | Test logic is separate from browser interaction code           |

## 5.2 Example: LoginPage class

```
class LoginPage(BasePage):
    EMAIL_INPUT = (By.ID, "input-email")
    PASSWORD_INPUT = (By.ID, "input-password")
    LOGIN_BUTTON = (By.CSS_SELECTOR, "input[value='Login']")
    ERROR_MESSAGE = (By.CSS_SELECTOR, "div.alert-danger")

    def login(self, email, password):
        self.enter_text(self.EMAIL_INPUT, email)
        self.enter_text(self.PASSWORD_INPUT, password)
        self.click_element(self.LOGIN_BUTTON)
```

## 5.3 BasePage: Reusable Base Class

All page classes inherit from BasePage, which provides common methods:

- `click_element(locator)` — Scrolls into view, waits for clickability, then clicks
- `enter_text(locator, text)` — Clears field, types character by character for demo effect
- `get_element_text(locator)` — Returns visible text of an element
- `is_element_displayed(locator)` — Returns True/False for visibility check
- `take_screenshot(name_prefix)` — Captures and saves a screenshot with a timestamp

# 6. Test Cases

## 6.1 Master E2E Journey (test\_01\_master\_e2e.py)

All tests share a single browser session (session-scoped fixture) and run in serial order to simulate a real continuous user journey.

| Test ID | Test Name             | Type                   | Description                                                         | Expected Result              |
|---------|-----------------------|------------------------|---------------------------------------------------------------------|------------------------------|
| TC-01   | test_01_invalid_login | Negative / Data-Driven | Read fake credentials from CSV → Attempt login → Check error banner | Warning message displayed    |
| TC-02   | test_02_valid_login   | Positive               | Login with valid credentials                                        | "My Account" heading visible |

|       |                                 |             |                                                                             |                                      |
|-------|---------------------------------|-------------|-----------------------------------------------------------------------------|--------------------------------------|
|       |                                 |             | from config.ini<br>→ Check My Account page                                  |                                      |
| TC-03 | test_03_search_and_view_product | Data-Driven | Read product names from CSV → Search each → Navigate to MacBook detail page | MacBook title visible on detail page |
| TC-04 | test_04_add_to_cart             | Functional  | Set qty → Click Add to Cart → Wait for banner → Verify MacBook in cart      | MacBook appears in cart              |
| TC-05 | test_05_clear_cart_and_logout   | Cleanup     | Remove all cart items → Verify empty → Logout via URL                       | Cart empty; user logged out          |

## 6.2 Additional Tests

| Test File                  | Description                                                                                           |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| test_02_screenshot_demo.py | Demonstrates on-failure screenshot embedding in the HTML report                                       |
| unittest_login.py          | Negative login test written using Python's built-in unittest framework with its own browser lifecycle |

## 7. Data-Driven Testing

Test data is stored in CSV files under test\_data/ and read at runtime using a custom read\_data.py utility. This keeps test logic completely separated from test data.

### 7.1 login\_data.csv

```
username,password,expected_result
random_nonexistent_xyz987@fake.com,wrongpassword123,fail
```

Used in: test\_01\_invalid\_login — reads the first row as invalid credentials for the negative login test.

### 7.2 search\_data.csv

```
search_query
MacBook
iPhone
Samsung SyncMaster
```

Used in: test\_03\_search\_and\_view\_product — iterates over all rows and searches each product.

## 8. Key Features

---

### 8.1 Automatic Screenshot on Failure

A pytest hook in `conftest.py` intercepts every test result. If a test fails, it:

- Captures a screenshot via the active WebDriver
- Saves it to the `screenshots/` folder with a timestamp
- Encodes it as Base64 and embeds it directly inside the HTML report (no broken image links)

### 8.2 Session-Scoped WebDriver

The setup fixture in `conftest.py` is scoped to the session, meaning one browser window is opened at the start and closed only at the end. All 5 tests share this browser, simulating a real continuous user session.

### 8.3 Cross-Browser Support

Tests can be run on either Chrome or Firefox without changing any code:

```
pytest --browser=chrome    # default
pytest --browser=firefox
```

WebDriver Manager automatically downloads the correct driver binary for the installed browser version.

### 8.4 Centralized Configuration

All environment-specific values are stored in `config/config.ini`:

- Base URL of the application
- Browser name (chrome / firefox)
- Implicit and explicit wait timeouts
- Login credentials for the test account

No values are hardcoded in test files. Tests read values via the `ReadConfig` utility class.

### 8.5 Timestamped Logging

Every test run creates a new `.log` file in the `logs/` folder (e.g., `automation_2026_09_17_11_51_00.log`). Log entries include timestamp, log level, and message for full traceability.

## 9. Configuration

---

`config/config.ini`:

```
[application]
base_url = https://tutorialsninja.com/demo/

[browser]
```

```

browser_name = chrome
headless = false

[timeouts]
implicit_wait = 10
explicit_wait = 15

[credentials]
valid_email      = <test account email>
valid_password   = <test account password>

```

## 10. How to Run the Project

---

### Prerequisites

- Python 3.10+ installed
- Google Chrome or Firefox browser installed
- Internet connection (for WebDriver Manager to download drivers on first run)

### Steps

```

# Step 1: Install dependencies
pip install -r requirements.txt

# Step 2: Run tests (Chrome, default)
pytest

# Step 3: (Optional) Run on Firefox
pytest --browser=firefox

# Step 4: View results
# Open  reports/report.html  in a browser

```

Windows One-Click Demo:

```
run_demo.bat
```

Cleans old outputs → runs Pytest suite → runs Unittest suite → displays report path.

## 11. Outputs

---

### 11.1 Screenshots

| Screenshot File                           | Description                                         |
|-------------------------------------------|-----------------------------------------------------|
| 01_Invalid_Credentials_*.png              | Captured after failed login attempt (negative test) |
| 02_Search_Result_MacBook_*.png            | Search results page for MacBook                     |
| 02_Search_Result_iPhone_*.png             | Search results page for iPhone                      |
| 02_Search_Result_Samsung_SyncMaster_*.png | Search results page for Samsung SyncMaster          |
| 03_Shopping_Cart_With_Item_*.png          | Shopping cart with MacBook added                    |
| test_99_screenshot_on_failure_demo_*.png  | Auto-captured on intentional test failure           |

## 11.2 HTML Report

A fully self-contained HTML report is generated at reports/report.html after every run. It includes:

- Pass / Fail status for every test case
- Test duration and execution time
- Captured log output per test
- Embedded failure screenshots (Base64-encoded, no external dependencies)

## 12. Challenges Faced & Solutions

| Challenge                                                    | Solution                                                                                                      |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Cart showed as empty after adding item (caching issue)       | Added wait_for_cart_to_update() to wait for cart counter to show "1 item(s)" before navigating                |
| StaleElementReferenceException when clearing cart in a loop  | Wrapped remove loop in try/except StaleElementReferenceException and re-fetched buttons each iteration        |
| unittest and pytest running together caused driver conflicts | Excluded unittest.TestCase subclasses from the autouse inject_driver_and_pause fixture using issubclass check |
| HTML report image links breaking when report is moved        | Switched from file-path image links to Base64-encoded embedded images inside the HTML                         |

## 13. Conclusion

This project successfully implements a complete, production-style web automation framework for an e-commerce application. The framework covers:

- 5 automated end-to-end test cases covering the full user journey
- Data-driven testing with CSV inputs (login and search)
- Negative and positive test scenarios
- Automatic failure screenshot embedded in the HTML report
- Cross-browser execution support (Chrome & Firefox)
- Clean layered architecture following the Page Object Model

The framework is designed to be easily extendable — new pages, test cases, or data files can be added with minimal changes to existing code. It reflects real-world industry practices used in professional QA automation teams.